

Ungoverned Writer: A Standalone Formalization of the Anti-Pattern in Which Adjacent Components Write Directly to Substrate Outside Cell-Rule Mediation in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 6, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, as a standalone anti-pattern, the failure mode in which adjacent AI components — RAG indexers, vector databases, integration platforms, webhook handlers, external AI services, cron jobs, third-party SaaS connectors — write directly to substrate without going through cells under orchestration rules, simultaneously violating the AI-as-substrate-mediator commitment (§4.2) and the hybrid-systems-composition commitment implicit in §2.3 and §6.1–§6.2 of the source paper.

Abstract

The CKS pattern locates substrate writes architecturally inside cells operating under human-authored orchestration rules: this is the operational content of AI-as-substrate-mediator (§4.2) and the boundary that hybrid composition with adjacent AI components (§2.3, §6.1, §6.2) must respect. A companion derivation note formalizes the three legitimate composition positions an adjacent component may occupy and names three corresponding anti-patterns. This note formalizes one of those, **ungoverned writer**, as a standalone failure mode with its own operational test, sharpening properties, and architectural correction. The note states the anti-pattern's four operational components, identifies the CKS commitments it violates, traces the failure mode it produces, specifies the architectural correction, distinguishes it from four adjacent legitimate patterns, and provides a three-property operational test. The standalone treatment intentionally duplicates content from the broader composition decomposition for defensive-publication purposes.

1. Why ungoverned writer needs to be formalized as a standalone anti-pattern

The source paper commits, in §4.2, to AI-as-substrate-mediator: substrate writes happen through cells operating under human-authored orchestration rules. It commits, in §2.3 and §6.1–§6.2, to hybrid composition with adjacent AI components — RAG indexes, fine-tuned models, vector databases, and external structured stores. A companion note formalizes the three legitimate composition positions an adjacent component may occupy — Pattern A consultation, Pattern B derived view, Pattern C separate concern — and names three corresponding anti-patterns. Ungoverned writer is the first.

A standalone publication is warranted because ungoverned-writer is the operationally most common failure mode in 2024–2026 hybrid AI deployments. Integration platforms with "write to database" actions, RAG indexers pushing embeddings into shared stores, vector databases with auto-update features, webhook handlers updating records on event receipt, cron-based jobs mutating data on schedule, AI agent frameworks with direct-write tooling, and "low-code/no-code" platforms whose default integration pattern is direct write all produce ungoverned writers when applied to coordination state. The drift is invisible at the moment of decision because direct-write integration *operationally works* — the substrate fills with content, the dashboards update, the automations fire. What is hidden is that the writes have no orchestration rule behind them, no cell mediating them, and no provenance attributing them to a rule and a cell. The standalone formalization provides operational test, sharpening properties, and architectural correction specific to this failure mode. The duplication with the composition decomposition is intentional: defensive publication of public prior art benefits from the same failure mode appearing in two independent landing sites.

2. The anti-pattern, defined precisely

A deployment exhibits the **ungoverned-writer** anti-pattern when its substrate is written to by components that are not cells operating under orchestration rules. The anti-pattern has four operational components, each of which independently constitutes the failure mode and which commonly co-occur.

(a) *Adjacent components performing substrate writes outside cell mediation.* The deployment includes one or more components that are architecturally adjacent to the substrate — neither cells nor part of the substrate itself — and that write directly to substrate state without a cell between them and the substrate. The architectural shape is *adjacent component* → *substrate*, lacking the cell-mediation that AI-as-substrate-mediator requires.

(b) *RAG indexers or vector databases pushing content directly to substrate.* The deployment runs a retrieval or embedding pipeline that writes its outputs — document chunks, embeddings, summaries, extracted entities — into substrate state without a cell processing them under a rule. Indexing pipelines that write extraction results as substrate content, vector databases with auto-update features that push embeddings into substrate fields, and retrieval systems whose ingest path terminates at substrate writes are common instantiations.

(c) *Integration platforms or webhook handlers writing substrate without rule authorization.* The deployment uses an integration platform or webhook handler whose action set includes "write substrate," and the writes fire on platform-defined triggers — schedule, event receipt, external state change — rather than under cell-rule mediation. The platform's "update record" action becomes a direct path into substrate state.

(d) *External AI services or automation tools writing substrate as their primary integration mode.* The deployment integrates an external AI service, automation tool, or third-party SaaS application whose primary integration with the substrate is to write into it — output of an external classifier landing as substrate content, output of an external agent landing as substrate decisions, scheduled job updating substrate state as its operational contract.

A deployment exhibiting any one of (a)–(d) partially exhibits the anti-pattern; a deployment exhibiting all four exhibits it fully. Multiple ungoverned writers acting concurrently produce emergent substrate state shaped by their interaction, with no rule specifying how they should interact and no cell mediating between them.

3. Which CKS commitments are violated

The anti-pattern violates two foundational architectural commitments simultaneously through the same operational mechanism.

AI-as-substrate-mediator (§4.2) is directly violated. The mediator role commits LLM-driven substrate writes to operate through cells under orchestration rules; the mediator decomposition's Property B — LLM writes under orchestration rules — is the specific commitment most directly violated for AI-driven adjacent components, and the mediator role's integrating frame fails for any adjacent component, AI-driven or not, that occupies the writer position.

Hybrid systems composition is directly violated. The composition commitment supports three positions — Pattern A consultation, Pattern B derived view, Pattern C separate concern. "Writer" is not one of these. Pattern A has the cell consulting the adjacent component and writing substrate under its rule; Pattern B has the adjacent component as a regeneratable derived view that does not write substrate at all; Pattern C has the adjacent component handling a non-coordination concern that does not couple to substrate. An adjacent component that writes substrate has acquired a position the architecture does not name. The composition decomposition's anti-pattern enumeration names this case explicitly as ungoverned writer.

The cascade across other commitments is direct. *Rule authoring as the canonical governance moment* is bypassed systematically: rules do not govern ungoverned writers because the writers operate outside the cell-rule architecture, and the architectural promise that humans govern through rule authorship loses traction at every ungoverned-write moment. *AI-as-mediator at every layer*, the requirement within composition requirements that mediation hold across composed substrates, is directly violated at the composition layer; *human-selective composition* is extended-violated when ungoverned writers operate outside the human-selected composition arrangement. *Path retraceability* is extended-violated: ungoverned writes typically lack rule attribution because no rule authored them, and the retraceable trail has gaps where the rationale collapses to "the integration platform did this" rather than "rule R authorized cell C." The six-field provenance the path-retraceability decomposition names — particularly the rule/orchestration reference field — is partially or fully violated when ungoverned writers do not record full provenance, which produces non-addressable-write instances at the same time. *Human-governed* is extended-implicated: humans exercising the modify right by modifying rules cannot reach ungoverned writers through rule-modification because the writers are not rule-mediated.

The pattern of the violation is uniform. Each commitment fails because the writer position is reserved, by the architecture, for cells under rules, and adjacent components occupying that position cannot satisfy the commitments the position carries.

4. The failure mode

Ungoverned writer produces deployments in which substrate state is shaped by writes that no rule authored.

Humans cannot govern substrate state changes through the architecture's rule-mediation mechanism. The human-governed commitment operates through rule authoring; humans modify rules to change cell behavior. Ungoverned writers do not consult rules, so the governance mechanism is operationally bypassed for their effects. A human who wants to change what the integration platform writes must change the integration platform's configuration, not a rule.

Writes lack rule attribution. The provenance field for the rule that authorized the write is missing or generic; "the integration platform wrote this," "the webhook fired," "the cron job ran" do not name a rule. The retraceable trail (§3.1) has gaps at every ungoverned-write moment, compounding with non-addressable-write instances when full provenance is absent.

Substrate state diverges from rule-governed expectations. A reviewer asking "what rule produced this content?" finds, for ungoverned writes, that no rule did. Multiple ungoverned writers operating concurrently produce emergent substrate state shaped by interactions that no rule mediates; two ungoverned writers writing conflicting content produce a conflict that no rule resolves, and the architectural commitment to rule-mediated conflict handling fails operationally.

Recovery from problematic writes is operationally constrained. The architectural recovery path is to modify a rule and let cells operate under the modified rule. Ungoverned writes have no rule to modify; recovery requires either modifying the external system that produced the writes — which may be outside deployment control — or accepting the substrate state. Both options sit outside the architecture's recovery commitments.

The "just works" framing masks the architectural failure. Deployment teams adopt integration platforms because the integrations operate smoothly and produce substrate state without bespoke engineering work, and the operational ease makes it harder to recognize that the architectural commitments have broken. The architectural commitment is that "just works" must mean "operates within the cell-rule architecture." This is the most consequential property of the anti-pattern: it is invisible at the moment a deployment decision is made, because the integration produces the substrate content the team wanted.

The anti-pattern compounds with adjacent failures. Ungoverned writers typically produce non-addressable-write instances because they do not record full provenance. AI-driven ungoverned writers compound with LLM-as-source-of-truth when the AI service's output becomes substrate content treated as authoritative. External tools acting as ungoverned writers compound with external-tool-state-treated-as-authoritative when the writes mirror external tool state into substrate.

5. The architectural correction

The architectural correction operates through three foundational commitments held together: cell-mediation for all substrate writes, adjacent components in legitimate composition positions, and rule authoring as the canonical governance moment.

Cell-mediation for all substrate writes. The architectural pattern for any substrate write is *rule authorizes cell → cell reads substrate → cell processes input under rule → cell writes substrate with attribution*. Adjacent components do not occupy the writer position; the writer position is reserved for cells under rules. Where a deployment previously had adjacent components writing substrate, the correction re-architects the integration so that a cell, authorized by a rule, reads from the adjacent component and writes substrate.

Adjacent components in Pattern A, B, or C positions. RAG indexers and vector databases that previously wrote substrate become Pattern B derived views or are consulted by cells under Pattern A. Integration platforms that previously had "write substrate" actions are re-architected so that the platform invokes a cell — the workflow-engine-triggers-cell pattern — and the cell reads from the platform's source and writes substrate under its rule. Webhook handlers become triggers for cells, not direct writers; the handler fires and a rule-authorized cell executes, reads the webhook event, and writes substrate. Cron jobs become rule-authorized cell invocations on schedule. External AI services are consulted by cells under Pattern A, with the cell processing the AI output under its rule and writing substrate.

Write-path-cell-mediation as an operational invariant. A correctly architected deployment audits substrate writes operationally to verify that all writes flow through cells under rules. The audit traces write origins; ungoverned writes are detectable because their origin is not a cell. Where the host environment supports it, the write-path infrastructure rejects writes that do not originate from cells; where it does not, the deployment relies on review of integration configurations to surface ungoverned-write paths. Cell-mediated writes that replace ungoverned writes record full six-field provenance, including the rule that authorized the cell, addressing the non-addressable-write compounding the anti-pattern produced.

The correction may require operationally significant work for deployments with extensive integration platform usage. A deployment that has built coordination state on top of dozens of integration-platform actions, RAG indexing pipelines, and webhook handlers may need to re-architect each of those paths. The correction does not promise that the work is small; it promises that the work, once done, restores the architectural commitments the deployment claimed.

6. What ungoverned writer is NOT

Four adjacent legitimate patterns are commonly conflated with ungoverned-writer; distinguishing them is what makes the anti-pattern operationally useful.

Not cell-mediated writes that drew on adjacent-component consultations. A cell that consults an adjacent component during execution and writes substrate based on rule processing of the consultation result is operating in Pattern A and is legitimate. The architectural shape is *cell consults adjacent component → cell processes under rule → cell writes substrate*; the adjacent component is consulted, not a writer.

Not cell-mediated mirroring of external state. Where a deployment needs substrate to reflect external state — a customer record, a document in an external store, a status from an external system — the legitimate pattern is a cell, authorized by a rule, that reads from the external source and writes substrate

under its rule. Direct external-source-to-substrate writing is the anti-pattern; cell-mediated mirroring is not.

Not Pattern C separate concerns operating without substrate coupling. External tools used for non-coordination concerns — sending email, processing payment, handling logging — that do not write substrate are legitimate Pattern C separate concerns. The anti-pattern arises when a Pattern C component acquires a substrate-write action and crosses into the writer position.

Not adjacent components as informational sources consulted by cells. Databases providing reference data, services providing computations, APIs providing external data — when consulted by cells under Pattern A and feeding cell-mediated writes — are legitimate. The anti-pattern arises when the same components write substrate directly rather than being consulted.

The four distinctions share a structure: in each case, a cell mediating between the adjacent component and substrate is what makes the configuration legitimate, and removing the cell is what makes it the anti-pattern.

7. Operational test

A deployment exhibits ungoverned-writer if any of the following are true at any time during the deployment's existence:

1. Adjacent components — RAG indexers, vector databases, integration platforms, webhook handlers, external AI services, cron jobs, third-party SaaS connectors — write directly to substrate without cell mediation.
2. RAG indexers or vector databases push content to substrate without cell processing under a rule.
3. Integration platforms include substrate-write actions that fire without rule authorization.
4. Webhook handlers, cron jobs, or external AI services write substrate as their primary integration mode.

Three sharpening properties operationalize the test for deployment review.

Write-path-cell-mediation test. Trace the origin of substrate writes; verify that every write originates from a cell. Writes from adjacent components without a cell between the component and the substrate indicate the anti-pattern.

Adjacent-component-position-classification test. For each adjacent component in the deployment, classify its position as Pattern A consultation, Pattern B derived view, or Pattern C separate concern. A component that does not classify into one of the three positions — typically because it writes substrate — indicates the anti-pattern. The test surfaces components that have drifted into the writer position even if individual writes look ordinary.

Write-rule-attribution test. Examine substrate writes for rule attribution; verify that the rule/orchestration-reference provenance field carries a specific rule reference. Missing or generic rule attribution indicates the anti-pattern, commonly compounding with non-addressable-write instances when full provenance is absent.

A deployment that fails any of (1)–(4) and any of the three sharpening properties exhibits the anti-pattern.

The one-sentence test: if a deployment has adjacent components writing directly to substrate without going through cells under orchestration rules, the deployment exhibits ungoverned-writer; the AI-as-substrate-mediator and hybrid-composition commitments both fail through the same mechanism, with rule authoring bypassed systematically and the AI-as-mediator-at-every-layer requirement directly violated.

8. Why naming ungoverned writer as a standalone anti-pattern matters

Implementations under pressure to deliver AI products with extensive integration capabilities consistently default to ungoverned-writer, because the integration tooling available to deployment teams in 2024–2026 is built around direct-write integration. Integration platforms ship with "write to database" actions; RAG and vector-database tools ship with direct-write APIs; webhook patterns commonly write to data stores on event receipt; cron-based automation writes directly by default; AI agent frameworks commonly include direct-substrate-write capabilities. The drift toward the anti-pattern is the path of least resistance through the available tools. A team that adopts these tools and applies them to coordination state without preserving cell-rule mediation produces ungoverned writers as a side effect of using the tools as documented.

The cost of the drift is that the architectural commitments degrade silently. Substrate state fills with writes no rule authored, the retraceable trail develops gaps, recovery operates outside the rule-modification mechanism, multiple ungoverned writers produce emergent substrate state, and the human-governed commitment loses operational reach. None of this is visible at the moment the integration is configured, because the integration produces substrate content the team wanted.

Naming ungoverned-writer as a standalone anti-pattern gives downstream readers a precise specification of the failure mode and its correction, citable independently of the broader composition decomposition. The duplication between this note and the composition decomposition is intentional for defensive-publication purposes. This note opens the composition-layer anti-pattern sub-cluster, which together with the formalizations of hidden bidirectional coupling and adjacent-component-as-substrate-substitute closes the three composition anti-patterns named in the composition decomposition.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Ungoverned Writer: A Standalone Formalization of the Anti-Pattern in Which Adjacent Components Write Directly to Substrate Outside Cell-Rule Mediation in the Coordination Knowledge Substrate Pattern*. May 6, 2026. ORCID: 0009-0004-8065-3235.